

Building a Solver for a Standard Constraint Puzzle — a Playbook

This repo now ships five independent Sudoku solvers across four algorithmic paradigms (see [SOLVERS.md](#) for the Sudoku-specific write-up of each). This document captures what generalizes from that process — the parts that would apply just as well to N-Queens, Kakuro, KenKen, a scheduling puzzle, or any other "fill in values subject to constraints" problem — as a reusable playbook for the *next* one.

1. Model the puzzle before picking an algorithm

Every solver here answers the same three questions before any code gets written:

- **Variables** — what are you deciding? (Sudoku: the digit in each cell.)
- **Domains** — what can each variable be, before any constraint narrows it?
- **Constraints** — what has to hold between variables? Sudoku's constraints all happen to be a single reusable shape, `AllDifferent(cells in a unit)`, which is *why* it admits so many different clean formulations. A puzzle built from more heterogeneous constraints (sums, inequalities, adjacency) will narrow which paradigms below fit well — see §3.

Get this model right first. Every solver in this repo — regardless of paradigm — implements the *same* CSP; only the algorithm changes. If you find yourself special-casing the puzzle's rules differently in every new solver, the shared model wasn't factored out clearly enough.

2. Reuse the primitives, don't re-derive them

`sudoku-engine.js` owns the puzzle-domain vocabulary — `emptyGrid` , `cloneGrid` , `gridsEqual` , `SIZE` , `BOX` — and every solver added since (`cp-solver.js` , `dlx-solver.js` , `milp-solver.js`) imports from it rather than re-declaring grid helpers or recomputing what a "box" is. When adding a solver for a new paradigm, resist rebuilding domain plumbing that already exists; import it. This also means each new solver file is almost entirely *algorithm*, with no incidental duplication to review or drift out of sync.

3. Choosing a paradigm

Four genuinely different strategies came up here, plus two that were considered and deliberately skipped. Roughly in order of "try this first":

Paradigm	What it does	Reach for it when	Cost to build
Backtracking + MRV	Depth-first search, branch on the most-constrained variable, undo on dead end	Almost always your first solver — simple, fast enough for puzzle-sized state spaces, easy to verify by hand	Low
Constraint propagation (CP)	Explicit per-variable domains, narrowed by logical inference rules to a fixpoint; search only when propagation stalls	The constraints have exploitable <i>local</i> structure (a value forced out of a domain cascades) — and you want the solve trace to double as a human-readable explanation	Medium
Exact cover (Dancing Links / Algorithm X)	Reformulate as "choose rows that jointly cover every constraint column exactly once"; solve with backtracking accelerated by $O(1)$ cover/uncover pointer surgery	Every constraint reduces to "exactly one of these choices must be true" (no sums, no inequalities) — this is a narrower fit than CP/backtracking but <i>very</i> fast when it applies	Medium (the linked-list bookkeeping is fiddly to get right, see §5)
MILP / ILP	Encode as linear (in)equalities over integer/binary variables, hand off to a real solver (this repo vendors HiGHS via WASM)	The constraints include genuine linear arithmetic (sums, weighted totals, inequalities) that the other three paradigms can't express directly	Low to build the model, but you're now depending on an external solver library
SAT (DPLL/CDCL)	Encode as CNF, solve with unit propagation + conflict-driven clause learning	Constraints are naturally boolean/logical and propagation-heavy, but you need something more general than exact cover	High — a correct CDCL implementation is substantial, and for puzzle-sized problems it rarely beats CP or DLX enough to justify it
Local search (simulated annealing,	Heuristically minimize constraint violations from a random starting point	Almost never, for puzzles with a small fixed solution space	Not worth it here — it isn't exact : it can stall or return no answer even when one

Paradigm	What it does	Reach for it when	Cost to build
genetic algorithms)			exists, which breaks any "cross-check every solver against the known solution" testing strategy (§6)

A recurring, load-bearing detail: **backtracking, CP, and DLX all use the same "pick the most-constrained thing first" heuristic**, just applied to a different unit — MRV picks the cell with fewest legal digits, CP's search step does the same after propagation, and DLX's "S heuristic" picks the constraint *column* with fewest candidate rows. If your new paradigm has an analogous choice point, apply the same idea there by default.

Two formulations can share the same underlying math and still perform wildly differently.

`solveGridMILPBinary` and `solveGridDLX` solve the *literal same* exact-cover matrix — one relaxes it into an LP and calls a general branch-and-cut solver, the other walks it directly with a purpose-built data structure. The direct approach was ~100x faster in this repo's measurements. Don't assume "it's the same formulation" means "pick whichever's easier to write" — the algorithm on top of the formulation is often the whole story.

4. Implementation lessons

- **Bitmask domains pay for themselves** in any propagation-heavy solver. `cp-solver.js` represents each cell's candidate set as a 9-bit int instead of a `Set` /array; membership, removal, and "is this a singleton" all become single machine instructions instead of allocations and scans.
 - **A given/fixed value doesn't need special-case code — encode it as a restricted domain instead.** CP starts a clue's domain as a singleton bitmask; DLX generates only *one* candidate row for a given cell instead of nine. Both let the normal algorithm handle it with zero extra branches — much more robust than writing a separate "handle the pre-filled cells" code path that has to stay in sync with the general case.
 - **Verify big-M constants (and any other magic numbers) against a known answer independently,** don't just trust the algebra. The integer MILP formulation's big-M was off by one in an earlier draft (`M=8` instead of the required `M=9`) — caught only by cross-checking output against a known-correct solution, not by re-reading the constraint math.
 - **Recursion depth is a non-issue at puzzle scale.** Every solver here recurses (search, propagation cascades, DLX cover/uncover) with no explicit depth guard, because the state space is small enough that it never matters in practice — confirmed empirically (§6) rather than assumed. Don't over-engineer an iterative rewrite pre-emptively; measure first.
-

5. Paradigm-specific gotcha: Dancing Links correctness

DLX's `cover()/uncover()` pointer surgery is easy to get subtly wrong in a way that *usually* still produces correct output — until it hits an adversarial input. The trap that came up directly during this project:

Don't try to pre-select rows for given/fixed values by manually calling `cover()` on their columns before search starts. If two givens conflict (e.g. corrupted input with two 5s in the same row), covering the first given's columns can — as a legitimate side effect — already remove the second given's row from the structure. Manually re-covering that row's columns afterward double-covers an already-covered column, corrupting the linked list in a way that doesn't throw, just silently produces wrong results.

The fix (used in `dlx-solver.js`): never special-case selection at all. Encode a given value as *only ever generating one candidate row* for that cell (§4 above), then let the ordinary `search()` loop discover and select it like anything else. Contradictions between givens then surface exactly the way any other dead end does — a column's candidate count reaches zero — with no separate code path to get wrong.

The general lesson: **when a paradigm has an invariant (here: "nothing is ever truly deleted, only spliced out and later spliced back") make sure every code path — including your "shortcut" for special input — goes through the same primitive operations that maintain it, rather than hand-rolling a shortcut that looks equivalent.**

6. Testing methodology that generalizes

None of these solvers were trusted based on "the logic looks right." Every one went through the same checklist before being wired into the app, regardless of paradigm:

1. **One worked example with an independently known answer.** A classic, widely-published example puzzle with a solution you can find elsewhere and paste in literally — not something the puzzle generator produced.
2. **Fuzz test against the generator's own ground truth.** Generate many instances (this repo: 20–100 per difficulty tier) with the app's existing generator, which independently verifies uniqueness, and compare the new solver's output to *that* generator's known-correct solution with a plain equality check.
3. **Cross-check against an already-trusted solver**, not just the known answer — e.g. DLX vs. backtracking on 30 fresh hard puzzles, agreeing every time. Two independent implementations agreeing is stronger evidence than either one agreeing with a single fixed example.
4. **Explicit infeasibility case.** Feed it an input with no solution (contradictory givens) and assert it correctly returns "no solution" rather than garbage or an infinite loop.
5. **Explicit under-constrained case.** Feed it an input with *multiple* valid solutions and assert it returns *some* valid solution — checked by validating the Sudoku rules directly, not by equality against one canonical answer, since exact/complete solvers are not required to agree with each other here.
6. **Benchmark with a warm-up phase and a large-enough sample.** A 20-sample run without JIT warm-up produced visibly noisier numbers than a 100-run warmed-up sample in this project; don't trust a handful of cold-start timings enough to put them in documentation.

Steps 1–5 are about correctness and are non-negotiable for an "exact and complete" solver claim. Step 6 is about honest performance claims — do both before writing a single comparison number into user-facing docs.

7. Environment/tooling notes (specific to a Node-less Windows box)

This repo has no Node.js and no build step by design (plain ES modules served statically). That constrains how a new solver gets *tested*, not just how it gets *written*:

- **Run ES modules in a real browser, headlessly, via Playwright**, rather than reaching for `node --experimental-modules` or similar — it isn't installed here, and the app's modules are meant to run in a browser anyway (`import.meta.url` -relative asset loading, etc., can behave differently under Node).
 - **Serve over HTTP, not `file://`** . Browsers block `<script type="module">` from loading over a bare `file://` URL (see README's "Running it"); spin up `python -m http.server` in the background before pointing Playwright at it.
 - **A locked file on Windows fails silently-ish** (`PermissionError` / `Device or resource busy`) if something else has it open (a PDF viewer, an editor tab). Write the new version to a fresh filename first, and only swap it into place once nothing else holds the original open — don't block the rest of the work on a lock that isn't yours to clear.
 - **Windows console encoding will choke on Unicode debug output** (`UnicodeEncodeError` on `cp1252` for characters like ✓) when printing Playwright-extracted page text back through Python's `print` . ASCII-encode with `errors='replace'` for debug prints; it doesn't matter for what actually ships (only for what you read while developing).
-

8. Documentation/integration conventions worth repeating

For consistency with what's already here, each new solver in this repo followed the same shape — a useful checklist for the next one:

1. `js/<name>-solver.js` — the implementation, with a header comment that explains the paradigm *and* explicitly contrasts it with the other solvers already in the repo (what's different, not just what it does).
2. `<name>-test.html` — a standalone, dependency-free smoke test against one known worked example, mirroring the existing `milp-test.html` / `cp-test.html` / `dlx-test.html` structure (not linked from `index.html`).
3. Wire into `js/app.js`'s `SOLVERS` array, the `results` initializer, and the comparison run (sync block for pure-JS solvers, the async `runX` pattern for anything WASM-backed) — the panel in `index.html` needs its "All N solvers..." copy updated too.
4. `SOLVERS.md` — a new numbered section (idea → formulation → pseudocode), renumbering everything after it, plus updates to the comparison table, the performance table (§7.2-equivalent), and the recommendation section. Regenerate `SOLVERS.pdf` from the updated markdown afterward — don't let the two drift.

Steps 3–4 are easy to half-do (add the solver but forget the copy that says "All N solvers...", or add a section but skip renumbering the ones after it). Treat the whole list as one unit of work, not "implement it" plus an optional follow-up.